

Final Capstone Proj

CAPSTONE PROJECT – FINAL REPORT TEMPLATE

(60%)

Cover Page

(Same format, report type: Final Report)

Abstract

Comprehensive summary of the completed project.

1. Introduction

- Project overview
- Objectives
- Organisational context

2. Improvement from Mid-Term (5%)

- Feedback received
- Changes implemented
- Improvements made

3. Development Process (35%)

3.1 Methodology Used

3.2 System Development / Implementation

3.3 Challenges and Solutions

3.4 Iterations and Enhancements

4. Project Outcomes (30%)

- Final system / product description
- Features implemented
- Screenshots / prototypes
- Testing and evaluation results
- Achievement of objectives

5. Project Completion (10%)

- Overall project status
- Limitations
- Future enhancements / recommendations

6. Poster & Presentation Summary (10%)

- Key highlights presented
- Contribution of the project

7. Conclusion

Overall reflection on project success and learning outcomes.

References

Appendices

(Optional)

- Detailed diagram
 - User Manuals
 - Code Samples
 - Supplementary materials

Criticism of last midterm report

- 2. Prob definition and 1.1 Problem statement are the same (just get rid of one and move on.
- Dont mention about the past proposals. Not a progress report
- Aims and Objectives are to build the archive system.
- Features of systems are not aims and objectives
- Skip Lessons learnt
- ERD was not clear, i need to get a better one
- Maybe put name/figure below the figure
- USE APA style referencing

Final Draft (point form)

Abstract

Comprehensive summary of the completed project.

- Project is completed
- All aims, objectives/features, done
- Can be deployed and bigger scale testing and evaluation can be done in the next 2 months
- AI-implementation failed to be realised.
- But possible

2. Improvement from Mid-Term (5%)

- Feedback received
 - From last presentation:

- Need more reference study on the tag system
 - Name may need to be reviewed : (Original proposed name: “Arkiba”)
 - Need more roles instead of just 2 (user and admin)
 - Security risk if every staff/teacher member has admin privileges
- Changes implemented
- Further research on Tag System may not be realised but instead
 - Added Staff:
 - Can delete files, Ban users, create, edit files. etc.
- Improvements made
- Renamed to “Arkiba:
 - Name is much more clearer
 - Does not sound close to other offensive/weird/worrying words in Malay or Arab
 - Close to Malay word archive
 - Instead we used an
 - Dynamic Folder System to categories Folders
 - ID-based materials paths,
 - Folders table to keep track of everything
 - Using virtual tree to output the folder structure in frontend UI

3. Development Process (35%)

3.1 Methodology Used

- Same as midterm : Agile
 - Use of AI to help speed things up and debug
 - Use of free opensource Vite components
 - List down main components used in code

3.2 System Development / Implementation

- In VS code
 - Using simple npm run dev and test.php files to test the logic and functionality of the .php files
 - [Screen shots of the testing] Or...
 - [Desc of what the test will upload]
-
- Dynamic FolderSystem took a while to set up
 - Most work in frontend UI logic

- Rendering of the File Grid and FileList
 - FileGrid is just a copy of FileList enhanced with help of AI

-

3.3 Challenges and Solutions

FRONTEND:

- Figuring what type of categorisation of each file/category to use: (as said earlier and in prev midterm report)
 - Either using Name-based - "physical" way
 - E.g: Creating a Biology folder called /biology/
 - If you rename that folder to "Life Sciences," the system has to physically move every single file inside it to a new location on the disk. If you have 10,000 archived documents, this is slow and risks "broken links."
 - So admin with access will have to hard-code it ourselves/create the folder within the archive files
 - Using Tree structure:
 - Original Folders table:
 - Folders:
 - Id
 - Name
 - Parent id
 - Type
 - Using recursive logic to "fetch" data: Tree structure
 - Frontend logic:
 - Uses <folder />
 - Renders self if it finds children
 - Put it in a flat storage and the categorisation will be handled by logic and new table
 - Materialised path:
 - Folder:
 - Id
 - Name
 - Path
 - No recursive: better performance
 - URL friendly
 - Alternative: Using the Flat Storage, Materialised Flat List but rendered using the recursive tree logic at frontend

-

- Path, will be split by '/' in treebuilder function. React will render in <folder>. The items listed by the render will show the names of files.
-
- Admin workflow for entering a new folder:
-
- **UI:** The admin is currently viewing **Math/2023/**.
- **Input:** They type "Winter" into a "New Folder Name" box.
- **Frontend/Backend Logic:** The system concatenates the current path with the new name:
Parent Path + New Name + / = Math/2023/Winter/
- **Database:** A new row is inserted with that exact path string.
- Why? Buildtree function does not have hardcoded names - future proof
- BUt:
 - Renaming may be hard;
 - If admin re-names, you have to change several rows. Find every single child that starts with the ori name and update.
 - So use...
 - `UPDATE papers SET path = REPLACE(path, 'Mathematics/', 'Maths/') WHERE path LIKE 'Mathematics/%';`
 -
 -
 - the path building is in backend
 - In upload: handle the path generation on the backend to ensure consistency.
 - Name transformation logic
- Or: the way we chose: ID-based materialised path
 - Renaming is instant
 - No broken links
 - Faster processing wise (though this should not matter in the matter of this small archive)

	D i s p l a y N a	
--	--	--

m e	
M a t h e m a t i c s	
2 0 2 3	
S u m m e r	

- So that users/staff can create the folders themselves once DB is populated.
- Only Update the path string when you are moving a folder. - then update all children.
- So we used: So...
- Folders: folderID, FolderName, pathIDString, parentID Nullable (FK Folders(folderID))
 - We then added type index idx_path to folders and index idx_folder (folderID) to archive table

- To make prefix search fast instead of scanning every row in folders table
 -
 - Added idx_path of type INDEX to table folders
- API
 - [Admin.js](#)
 - Functions for admins,
 - List Users down
 - SetUserStatus - bans unbans users
 - Easier as a toggle
 - getActivityLogs
 - For deashbaord
 - GetTopDownloads
 - Get top 10 from activity.php
 - [auth.js](#)
 - For user Login and authentication
 - Explain briefly each one
 - [File.js](#)
 - For getting, uploading,update, getting download url,
 - Added bulkUpload, Delete
 - Added Folder CRUD
- Components:
 - BulkUploadsModal.jsx
 - UI Component To allow bulk upload. Not a page because its easier/more intuitive to see files then uploading
 - Use SetMetafiled because because we wanted tomake it dynamic without affecting other files
 - Handlesubmit - explain
 - handledrop
 - Allow different types of files for papers
 - Allow all types of files fo photos
 - Filegrid.jsx and FileList.jsx
 - Component to output list of files of each folder
 - Show the card: delete, download etc.
 - Show if its deleted or not
 - Handle bulk delete and bulk restore with checklist feature
 - Set if a user is looking, hide the deleted files
 - Allow inline editing
 - Both Same things. Ill probably retire FileGrid (redundancy). Started with filegrid, but to make it more simpler I opted for a list
 - Used both for a users preference
 -
 - FolderTree
 - Rendering of Folder structure using...

- Handle folder CRUD
 - Handle of moving of folders
- Navbar.jsx
 - Simple navbar
- ProtectedRoute.jsx
 - Explain briefly
- Pages:
 - Maybe do explain the headers and search where i got them from
 - AccountPage.jsx
 - Login and update
 - Had to make it inline-editable for account updating
 - ActivityDashboard.jsx
 - Output as csv or pdf?
 - Opted for both
 - No time to refine further
 - Graphs etc.
 - Explained earlier
 - AdminUsersPage.jsx
 - Toggle Ban Usrs because why not
 - Handle admin view for user list
 - Toggle between roles
 - BrowseUsersPage.jsx
 - Yeah
 - ForgotPasword.jsx
 - LandingPage.jsx
 - Yeah
 - AI-chat/search is still under construction
 - LoginPage.jsx
 - Catpcha
 - RegisterPage.jsx
 - ResetPasswordPage.jsx
 - Upload.jsx
 - Maybe irrelevant
 -
-
-

- BACKEND:
 - Bulk Transfer/uploading
 - Deciding on whether a user/staff should be able to bulk upload or upload individual files one by one

- Decided on prioritising the ease of the user/staff by making bulk uploads possible
- Instead of an upload page, staff can submit and drag and drop files
- Bulk Naming (file Updating)
 - Since users are able to upload massive amounts of files at once, updating the month, year, semester, subject etc. of each file would be tiresome one by one.
 - So when uploading in bulk a staff will be able to put in the subject, month, year etc. of the subject of multiple at once
 - Added fileName nullable into papermetadata and photometadata. Staff can rename files without physically renaming the actual files
 - Else the default name shown in UI is just the FilePath after the last “\”
- Planning on making bulk updates: when a user uploads in bulk
- Dashboard reporting
 - Deciding on what to show
 - Activity Log
 - List down list of activity (sort by latest)
 - Graph of activity (default to past month)
 - Filter (by users, time (period) (type in), role, ActionType (LOGIN, LOGIN_FAIL, UPDATE, UPLOAD, BULK UPLOAD, DOWNLOAD, VIEW, REGISTER and user's own query)
 - Add multiple choice filter/checklist
 - Filter by folder name of the files (rank them by hierarchy of the files, parents top, least child folders bottom).
 -
 - I've added a targetFileID in logActivity:
 - TargetFileID INT NULL
 - Change logActivity under crud\, so that for DOWNLOAD, UPLOAD, UPDATE, DELETE actions will have their TargetFileID populated with the relevant fileID
 -
 - With that
 - Add a top 10 most popular (most downloaded) files. Printable to csv
 -
 - Report maker:
 - A csv report maker
 - Query then export (from filter above)
 -
 -
 - PDF: jsPDF or html2canvas that prints pdf the graph
- Authentication:
 - Deciding on using 3 types of users/roles: Admin (1), User (2), Staff (3)

- According to research this is about the best security can do with the Things i have:
 - PHP, secure cookies
 - Password_verify()
 - Session_start()
 - Store ID, role, email in \$_session
 - Login.php - takes.... verifies
 - Logout.php - destroys the session
 - Session.php
 - Auth.php
 - Password Hashing
 - password_hash()
 - Password_default
 - Password verify()
 - Role-based
 - \$_SESSION['roleID']
 - LogActivity for failed login
 - Deciding on Captcha:
 - Google ReCaptcha: 10,000 users per month
 - Captures a lot of data
 - Cloudflare turnstile: 1 million request
 - “Invisible” - may need to look up more on this
 - Javascript challenges, device finger printing
 - Lighter and faster
 - add a script tag to your React login page, include a div with your site key, and then verify the token on the PHP side with a simple POST to Cloudflare's API.
 - ALTCHA
 - Open-source, self hosted
 - No external dependencies
 - Less battle tested than google and cloudflare.
 - My approach:
 - Cloudflare turnstile
- CRUD
 - Initially it was just:
 - Create.php
 - Delete.php
 - SoftDelete process as explained in midterm report
 - Updated to add new role staff
 - Download.php
 - Update.php
 - Then:
 - Bulk_upload
 - Folders
 - LogActivity Logs everything into activity
 -
- Accounts
 - Login
 - Logout
 - Pass_forgot and pass_reset

- Using email to reset password:
 - Using phpmailer...
- Registration
 - Also using phpmailer to confirm email account
 - User makes and waits for admin/staff to confirm verify
 - Added isVerified in account table: 0 for not, 1 for yes
 - For testing reasons, isverified is commented out
- Update.php
- Verify_email
 - Not yet used/finished
- Tags:
 - Simple Tags Crud
 -
-
- Session security:
 - Fail login only a handful of times before being forced to wait 15 minutes for security
 - Based on IP-Adress
 - How to get IP address?
 -
- Admin
 - As part of dashboard creation
 - Activity.php
 - Returns activity data for admin dashboard
 - Users.php
 - For managing Users account ban/unban
 - Optimise for getting just users, just staff or just admin instead of just all
 - For frontend UI
 - More logical Toggleable ban/unban feature
 - LogActivity
- [Archive.js](#)
 - Explain
-

3.4 Iterations and Enhancements

4. Project Outcomes (30%)

- Final system / product description
- Features implemented
- Screenshots / prototypes

- Testing and evaluation results
- Achievement of objectives

5. Project Completion (10%)

- Overall project status
- Limitations
- Future enhancements / recommendations

6. Poster & Presentation Summary (10%)

- Key highlights presented
- Contribution of the project

7. Conclusion

Overall reflection on project success and learning outcomes.

References

Appendices

(Optional)

- Detailed diagram
 - User Manuals
 - Code Samples
 - Supplementary materials

